AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

DATA STRUCTURES ANDALGORITHMS

23CSE203

LAB MANUAL:

Submitted to:	
Name	**Dr. Rajkumar Batchu**
Department	**CSE**
Designation	**Assistant professor(Sl. Gd.)**

Submitted by:	
Name	**J. John shyam**
Roll No	**AV.SC.U4CSE25115**
Year/Sem/Section	**2nd YEAR/SEM-3/CSE-B**

INDEX:

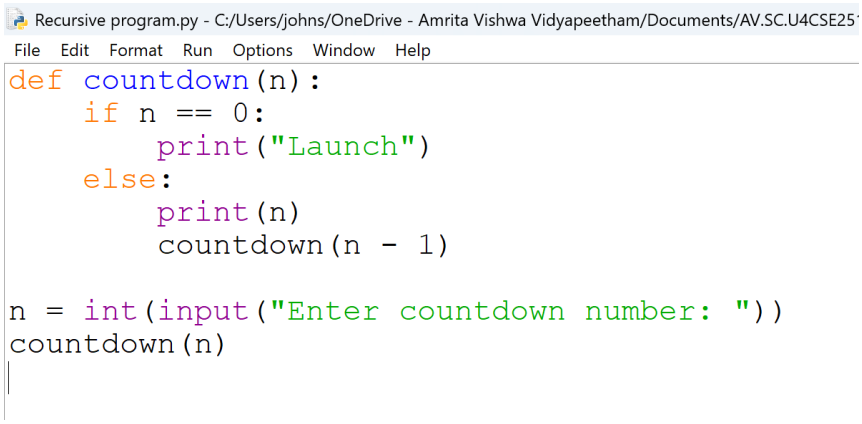
S.NO	TITLE / TOPICS	DATE	PAGE NO
1	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	14-07-26	4-11
2	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	21-07-26	12-18
3	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	01-08-26	19-30
4	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	11-08-26	31-45
5	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL)	18-08-26	46-52

	Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit		
6	**WEEK 6: Stacks** A) Implement Stack operations (Push, Pop, Peek, Display) using Array. B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	25-08-26	53-57
7	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	22-09-26	58-62
8	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	29-09-26	63-66
9	**WEEK 9:** **Binary Search Trees** Python program to implement Binary Search Tree (BST) operations.	06-10-26	67-69
10	**WEEK 10: AVL Tree** Write a Python program to implement an AVL Tree with all its operations.	13-10-26	70-74
11	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	27-10-26	75-79

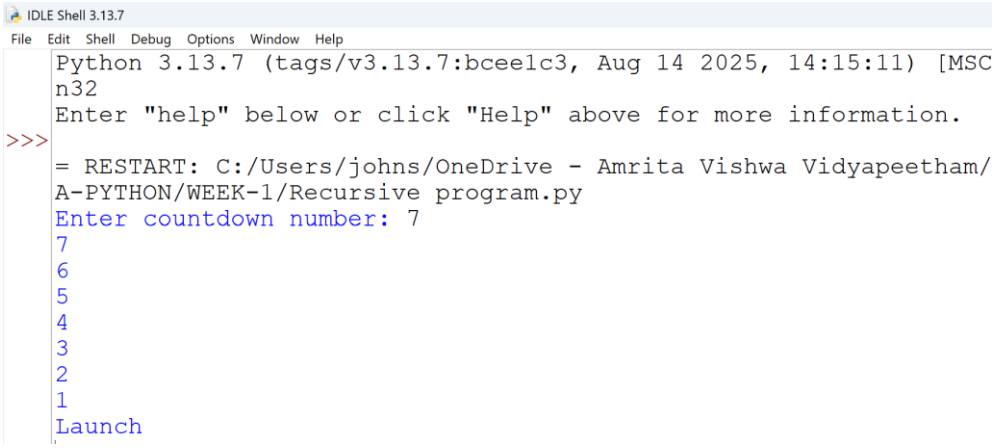
WEEK-1:

1A. AIM: A Rocket launch system displays a countdown before launch. Instead of suing a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by "launch".

Code:



Output:



Explanation

Condition 1: Positive Input (n > 0)

- The function checks whether n == 0.
- Since n is positive, the condition is **False**.
- It executes:
  - print(n) → prints the current value of n.
  - countdown(n-1) → calls the same function again with n-1.
- This process continues until n becomes 0.

- When n == 0, the condition becomes **True**.
- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

Condition 2: Zero Input (n == 0)

- The function checks if n == 0.
- The condition is immediately **True**.
- It prints **launch**.
- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

Condition 3: Negative Input (n < 0)

- The condition n == 0 is **False**.
- It prints the negative value.
- Then it calls itself with n-1, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.
- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

Error table:

s.no	Error obtained	Rectification
1	Negative input in countdown	Add a condition to handle negative numbers before recursion.

1B. Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p(n) where p=principal growth, n= no. of years.

Code:

```
File  Edit  Format  Run  Options  Window  Help
def compound_interest(principal, years):
    if years == 0:
        return 1
    return principal * compound_interest(principal, years - 1)


principal = int(input("Enter the principal growth: "))
years = int(input("Enter the number of years: "))

result = compound_interest(principal, years)
print(result)
```

Output:

```
= RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidya
A-PYTHON/WEEK-1/1B.py
Enter the principal growth: 5
Enter the number of years: 5
3125
```

Explanation:

Condition 1: n > 0

- The function compound_interest(p, n) is called with the principal amount p and number of years n.
- The function checks whether n == 0.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:
  - p * compound_interest(p, n-1)
- The function calls itself recursively with n-1.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.

- Therefore, the function calculates $\mathbf{p^n}$.

Condition 2: n == 0

- When n becomes 0, the condition n == 0 becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.
- The recursive calls then return one by one.
- Each returned value is multiplied by p.
- Finally, the result is printed using print(compound_interest(p,n))

Error table:

s.no	Error obtained	rectification
1	Wrong indentation	Maintain proper indentation.

1C. AIM . An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code:

3.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week1\3.py (3.13.7)

File Edit Format Run Options Window Help

```
def find_employee(employees, target_id, position):
    if position >= len(employees):
        return False

    if employees[position] == target_id:
        return True

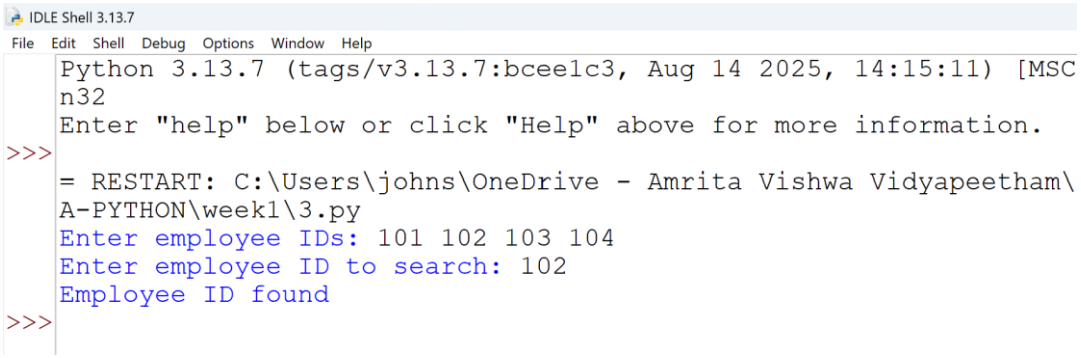
    return find_employee(employees, target_id, position + 1)


emp_list = list(map(int, input("Enter employee IDs: ").split()))
emp_id = int(input("Enter employee ID to search: "))

found = find_employee(emp_list, emp_id, 0)

if found:
    print("Employee ID found")
else:
    print("Employee ID not found")
```

Output:



Explanation

Condition 1: index == len(emp_list)

- The function checks whether index has reached the length of the employee list.
- If index == len(emp_list), it means we have checked **all employee IDs**.
- The employee ID was not found.
- The function returns **False**.
- No further recursion takes place.

Condition 2: emp_list[index] == emp_id

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

Condition 3: Employee ID does not match

- If the current employee ID is not equal to emp_id, the condition is **False**.
- The function calls itself recursively with:
  - index + 1
- This moves to the **next employee ID** in the list.
- The process continues until:
  - The employee ID is found, or

Error table:

s.no	Error obtained	Rectification
1	Missing return statement	Add the required return statement

1D. AIM. In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code:

```
4.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week1\4.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
def factorial(num):
    if num < 0:
        return "Factorial is not defined for negative numbers"

    if num == 0:
        return 1

    return num * factorial(num - 1)


n = int(input("Enter a number: "))

result = factorial(n)
print("Factorial =", result)
```

Output:

```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
    Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC
    n32
    Enter "help" below or click "Help" above for more information.
>>>
    = RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\D
    A-PYTHON\week1\4.py
    Enter a number: 9
    Factorial = 362880
>>>
```

Explanation

Condition 1: Negative Input (n < 0)

- The program first checks whether n < 0.
- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
  Factorial is not defined for negative numbers.

- The factorial() function is **not called**.

Condition 2: Base Case (n == 0 or n == 1)

- Inside the factorial() function, it checks:
  n == 0 or n == 1
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- It stops further recursive calls.

Condition 3: n > 1

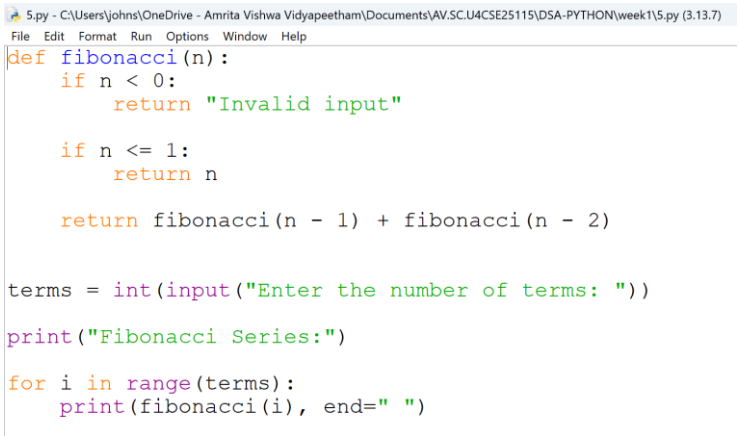
- If n is greater than 1, the condition n == 0 or n == 1 is **False**.
- It returns:
  n * factorial(n - 1)
- The function calls itself with n - 1.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

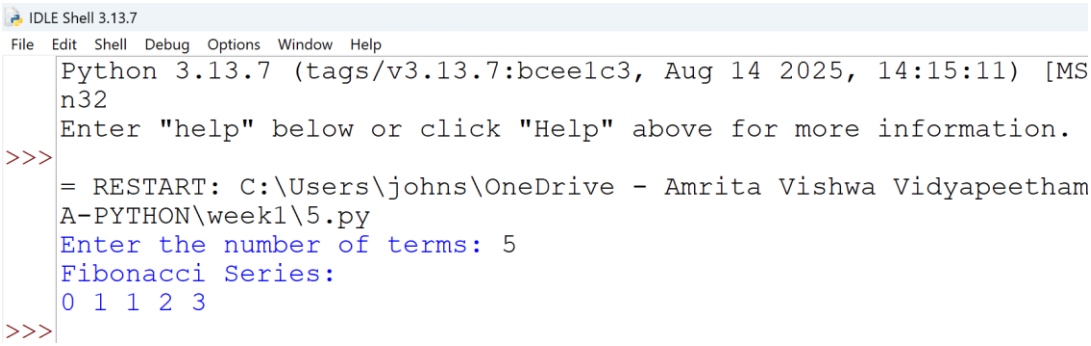
s.no	Error obtained	Rectification
1	Wrong recursive value	Updated the recursive value correctly

1E. AIM. Write a recursive program to print the first n terms of the fibonacci series

Code:



Output:



Explanation

Condition 1: n == 0

- The function checks whether n == 0.
- If the condition is **True**, the function returns 0.
- This is the **first base case**.

Condition 2: n == 1

- If n is not 0, the function checks whether n == 1.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.

Condition 3: n > 1

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.
- The function calls itself **two times**:
  - fibonacci(n - 1)
  - fibonacci(n - 2)
- This process continues until the base cases n == 0 and n == 1 are reached.

Error table:

s.no	Error obtained	Rectification
1	Incorrect Fibonacci recursion	Use fibonacci(n-1) + fibonacci(n-2).

WEEK-2:

2a. AIM: Write a python program to perform linear search on the give input.

Code:

```
1.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week2\1.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
def linear_search(numbers, target):
    for index in range(len(numbers)):
        if numbers[index] == target:
            return index
    return -1


n = int(input("Enter the number of elements: "))

numbers = []

for _ in range(n):
    numbers.append(int(input("Enter the element: ")))

target = int(input("Enter the search element: "))

result = linear_search(numbers, target)
print(result)
```

Output:

```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
    Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC
    n32
    Enter "help" below or click "Help" above for more information.
>>>
    = RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\D
    A-PYTHON\week2\1.py
    Enter the number of elements: 5
    Enter the element: 1
    Enter the element: 2
    Enter the element: 3
    Enter the element: 4
    Enter the element: 5
    Enter the search element: 6
    -1
>>>
```

Explanation

Condition 1: Element is Found (arr[i] == k)

- The function linear_search(arr, k) receives the array arr and the search element k.
- The for loop starts from the first element of the array.
- It checks each element one by one using:
  arr[i] == k
- If the current element is equal to k, the condition becomes **True**.

- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

Condition 2: Element is Not Found

- The loop checks every element in the array.
- If none of the elements is equal to k, the if condition is always **False**.
- After the loop finishes, the statement return -1 is executed.
- -1 indicates that the search element is **not present** in the array.

Error table:

s.no	Error obtained	Rectification
1	Wrong loop range	Use range(n) to process exactly n elements.

2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

Code:

2.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week2\2.py (3.13.7)

File Edit Format Run Options Window Help

```
def binary_search(numbers, target):
    left = 0
    right = len(numbers) - 1

    while left <= right:
        middle = (left + right) // 2

        if numbers[middle] == target:
            return middle

        if target < numbers[middle]:
            right = middle - 1
        else:
            left = middle + 1

    return -1


n = int(input("Enter the number of elements: "))

numbers = []

for _ in range(n):
    value = int(input("Enter the element: "))
    numbers.append(value)

target = int(input("Enter the search element: "))

position = binary_search(numbers, target)
print(position)
```

Output:

```
>
= RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\
A-PYTHON\week2\2.py
Enter the number of elements: 5
Enter the element: 1
Enter the element: 2
Enter the element: 3
Enter the element: 4
Enter the element: 5
Enter the search element: 2
1
>
```

Explanation

Condition 1: k == arr[mid] — Element Found

- The function binary_search(arr, k) receives the array and the search element.
- low is initialized to 0, which represents the first index.
- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
  mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

Condition 2: k < arr[mid] — Search Left Half

- If k is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
  high = mid - 1
- The right half is eliminated.
- The process continues with the remaining left half

Condition 3: k > arr[mid] — Search Right Half

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.
- This means the search element can only be present in the **right half**.

- Therefore:
  low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

Condition 4: low > high — Element Not Found

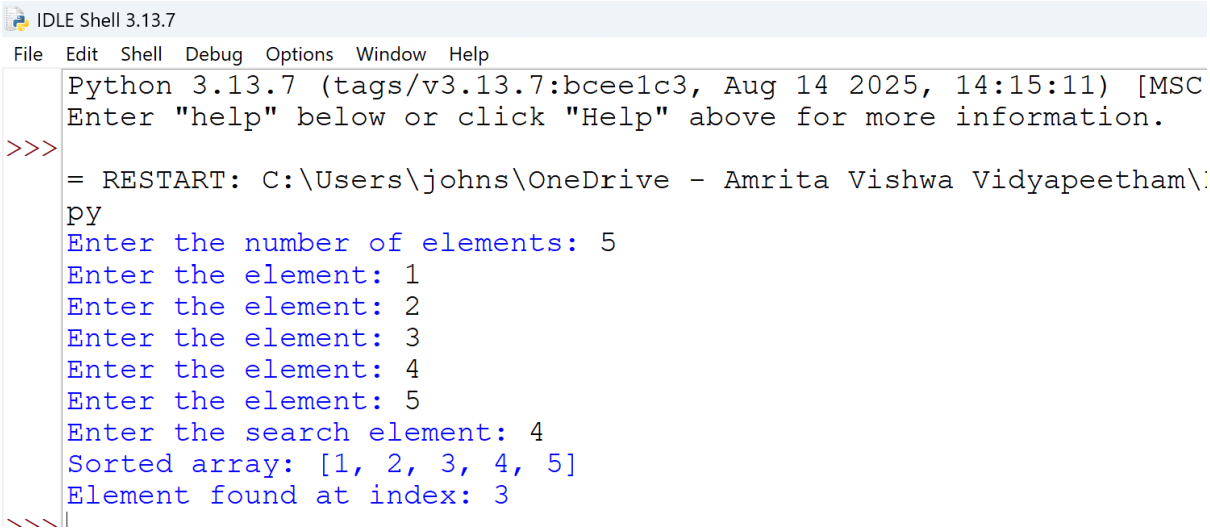
- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

s.no	Error obtained	Rectification
	Wrong index in binary search	Use high = mid - 1 for the left half and low = mid + 1 for the right half.

2c.Aim: write a python program to perform a bianry search on the given input verify by Taking unsorted array as input

Code:



3.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week2\3.py

File Edit Format Run Options Window Help

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif target < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1


n = int(input("Enter the number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input("Enter the element: ")))

target = int(input("Enter the search element: "))

# Sort the unsorted array
arr.sort()

print("Sorted array:", arr)

result = binary_search(arr, target)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Explanation

Condition 1: arr[mid] == key — Element Found

- The function first calculates the middle index using mid = (low + high) // 2.
- It compares the middle element arr[mid] with the search element key.
- If arr[mid] == key, the condition is **True**.
- The element is found.
- The function returns the index mid.
- The search stops.

Condition 2: key < arr[mid] — Search Left Half

- If key is smaller than arr[mid], the condition is **True**.

- This means the required element may be present in the **left half**.
- The right half is ignored.
- high is changed to mid - 1.
- The search continues in the left half.

Condition 3: key > arr[mid] — Search Right Half

- If key is greater than arr[mid], the previous conditions are **False**.
- The element may be present in the **right half**.
- The left half is ignored.
- low is changed to mid + 1.
- The search continues in the right half.

Condition 4: low > high — Element Not Found

- The while low <= high loop continues until there are elements left to search.
- When low > high, the loop stops.
- This means the element is not present in the array.
- The function returns -1.
- The program prints **"Element not found."**

Condition 5: Array is Already Sorted

- The program checks whether the given array is already sorted.
- It uses:
  arr == sorted(arr)
- If the condition is **True**, it prints:
  "Array is already sorted."
- The array is directly used for binary search.

Condition 6: Array is Unsorted

- If arr == sorted(arr) is **False**, the array is unsorted.
- The program prints:
  "Array is unsorted."
- It then uses arr.sort() to sort the array.

- The sorted array is displayed.
- Binary search is then performed on the sorted array.

Error table:

s.no	Error obtained	Rectification
1	Binary search on unsorted array	sort the array first using arr.sort() or provide a sorted array.

Week-3:

3A.Aim.Write a python program to sort the elements using a bubble sort technique. Take an input elements from the user.

Code:

1.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week3\1.py (3.13.7)

File Edit Format Run Options Window Help

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


n = int(input("Enter the number of elements: "))

arr = []

for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

bubble_sort(arr)

print("After sorting:", arr)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15
Enter "help" below or click "Help" above for more infor
>>>
= RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidy
py
Enter the number of elements: 5
Enter the element: 1
Enter the element: 2
Enter the element: 3
Enter the element: 4
Enter the element: 5
Before sorting: [1, 2, 3, 4, 5]
After sorting: [1, 2, 3, 4, 5]
>>>
```

Explanation

Condition 1: arr[j] > arr[j+1] — Elements Need to Be Swapped

- **The function bubble_sort(arr) receives the array as an argument.**
- **The outer for loop runs from 0 to n-1 and controls the number of passes.**

- **The inner for loop runs from 0 to n-2 and compares adjacent elements.**
- **The condition arr[j] > arr[j+1] checks whether the current element is greater than the next element.**
- **If the condition is True, the two elements are in the wrong order.**
- **Therefore, the two elements are swapped using:**
- **Arr[j],arr[j+1]=arr[j+1],arr[j]**
- **After the swap, the smaller element moves toward the left and the larger element moves toward the right.**

Condition 2: arr[j] <= arr[j+1] — No Swap Required

- **If arr[j] > arr[j+1] is False, it means the current element is smaller than or equal to the next element.**
- **Therefore, the elements are already in the correct ascending order.**
- **No swapping is performed.**
- **The inner loop continues to compare the next pair of adjacent elements.**

Condition 3: for i in range(n) — Repeated Passes

- **The outer loop repeats the comparison process for all the elements.**
- **After each complete pass, the largest unsorted element moves to the end of the array.**
- **The process is repeated until all elements are arranged in ascending order.**
- **Finally, the function returns the sorted array using:**

Error table:

sno	Error obtained	Rectification
1	IndexError: list index out of range	Use correct loop range

3b.Aim. write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code:

2.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week3\2.py (3.

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key


n = int(input("Enter the number of elements: "))

arr = []

for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

insertion_sort(arr)

print("After sorting:", arr)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 6
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents
py
Enter the number of elements: 4
Enter the element: 1
Enter the element: 5
Enter the element: 6
Enter the element: 7
Before sorting: [1, 5, 6, 7]
After sorting: [1, 5, 6, 7]
>>>
```

Explanation:

Condition 1: for i in range(1, len(arr)) — Selecting Element

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

Condition 2: key = arr[i] — Storing Element

- The current element is stored in key.

key = arr[i]

- This element will be compared with the elements before it.

Condition 3: j = i - 1 — Getting Previous Position

- j points to the element just before the current element.

```
j = i - 1
```

Condition 4: while j >= 0 and key < arr[j] — Comparing Elements

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

```
while j >= 0 and key < arr[j]:
    arr[j + 1] = arr[j]
    j -= 1
```

- This creates space for the current element.

Condition 5: arr[j + 1] = key — Inserting Element

- The key is placed in its correct position.

```
arr[j + 1] = key
```

- The elements before the current position are now sorted.

Condition 6: return arr — Returning Sorted Array

- After all elements are processed, the function returns the sorted array.

```
return arr
```

error table:

sno	Error obtained	Rectification
1	Elements not sorted correctly	Shift elements before insertion

3c.write a python program to sort the elements using selection sort technique . take an input from the user.

Code:

3.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        smallest = i

        for j in range(i + 1, n):
            if arr[j] < arr[smallest]:
                smallest = j

        arr[i], arr[smallest] = arr[smallest], arr[i]


n = int(input("Enter the number of elements: "))

arr = []

for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

selection_sort(arr)

print("After sorting:", arr)
```

Output:

```
= RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Docu
py
Enter the number of elements: 4
Enter the element: 9
Enter the element: 2
Enter the element: 5
Enter the element: 2
Before sorting: [9, 2, 5, 2]
After sorting: [2, 2, 5, 9]
>>
```

Explanation:

Condition 1: for i in range(n) — Selecting Position

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

Condition 2: mini = i — Assuming Minimum

- Initially, the element at index i is assumed to be the smallest.

Condition 3: for j in range(i+1,n) — Finding Minimum

- The loop checks all elements after position i.
- It searches for the smallest element.

Condition 4: if arr[mini] > arr[j] — Comparing

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

Condition 5: mini = j — Updating Minimum

- Stores the index of the newly found smallest element.

Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

Condition 7: return arr — Returning Result

- Returns the completely sorted array.

Error table:

Sno	Error obtained	Rectification
1	Wrong minimum value	Update min_index properly

3D.write a python program to sort the elements using the merge sort technique . take an input from the user.

Code:

5.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHON\week3\5.py (3.13.7)

File Edit Format Run Options Window Help

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    smaller = []
    greater = []

    for value in arr[:-1]:
        if value <= pivot:
            smaller.append(value)
        else:
            greater.append(value)

    return quick_sort(smaller) + [pivot] + quick_sort(greater)


n = int(input("Enter the number of elements: "))

arr = []

for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

sorted_arr = quick_sort(arr)

print("After sorting:", sorted_arr)
```

Output:

```
= RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vid
py
Enter the number of elements: 6
Enter the element: 3
Enter the element: 8
Enter the element: 2
Enter the element: 8
Enter the element: 2
Enter the element: 1
Before sorting: [3, 8, 2, 8, 2, 1]
After sorting: [1, 2, 2, 3, 8, 8]
```

Explanation:

Condition 1: if len(arr) <= 1 — Checking the Base Condition

- The function first checks whether the array contains **one or zero elements**.

```
if len(arr) <= 1:
    return arr
```

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

```
return arr
```

Condition 2: mid = len(arr) // 2 — Finding the Middle

- The array is divided into two parts.
- The middle position is calculated using:

```
mid = len(arr) // 2
```

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

Condition 3: left = arr[:mid] — Creating the Left Part

- The elements before the middle position are stored in left.

```
left = arr[:mid]
```

Condition 4: right = arr[mid:] — Creating the Right Part

- The elements from the middle position to the end are stored in right.

```
right = arr[mid:]
```

Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts

- The left and right parts are recursively sorted.

```
merge_sort(left)
```

merge_sort(right)

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

Condition 6: i = j = k = 0 — Initializing Indexes

- Three variables are initialized:

i = j = k = 0

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts

- The program compares elements from the left and right arrays.

while i < len(left) and j < len(right):

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

Condition 8: if left[i] < right[j] — Comparing Elements

- The current elements of the left and right arrays are compared.

if left[i] < right[j]:

- If the left element is smaller than the right element, it is placed into the original array.

arr[k] = left[i]

- Then i is increased:

i += 1

- This moves to the next element of the left array.

Condition 9: else — Taking the Right Element

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

else:

arr[k] = right[j]

- Then j is increased:

j += 1

- This moves to the next element of the right array.

Condition 10: k += 1 — Moving to the Next Position

- After placing an element into the original array, k is increased.

k += 1

Condition 11: while i < len(left) — Copying Remaining Left Elements

- Sometimes the right array becomes empty before the left array.

while i < len(left):

- Any remaining elements in the left array are copied into the original array.

arr[k] = left[i]

- Then both i and k are increased:

i += 1

k += 1

- The remaining elements are already sorted, so no comparison is needed.

Condition 12: while j < len(right) — Copying Remaining Right Elements

- Sometimes the left array becomes empty before the right array.

while j < len(right):

- Any remaining elements in the right array are copied into the original array.

arr[k] = right[j]

- Then j and k are increased:

j += 1

k += 1

- This completes the merging process.

Error table:

sno	Error obtained	Rectification
1	Recursive error	Use if len(arr) <= 1
2	IndexError	Check left/right array indices

3E. Aim. Write a python program to sort the elements using a quick sort technique . take an input elements from the user

Code:

5.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA

File Edit Format Run Options Window Help

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    smaller = []
    greater = []

    for value in arr[:-1]:
        if value <= pivot:
            smaller.append(value)
        else:
            greater.append(value)

    return quick_sort(smaller) + [pivot] + quick_sort(greater)


n = int(input("Enter the number of elements: "))

arr = []

for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

sorted_arr = quick_sort(arr)

print("After sorting:", sorted_arr)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\
Enter the number of elements: 4
Enter the element: 9
Enter the element: 1
Enter the element: 6
Enter the element: 3
Before sorting: [9, 1, 6, 3]
After sorting: [1, 3, 6, 9]
>>>
```

Explanation

Condition 1: if arr[j] < pivot — Element Smaller Than Pivot

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.

- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.
- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

arr[i], arr[j] = arr[j], arr[i]

- This places all elements smaller than the pivot toward the left side.

Condition 2: arr[j] >= pivot — No Swap Required

- If arr[j] < pivot is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

Condition 3: arr[i+1], arr[high] = arr[high], arr[i+1] — Placing the Pivot

- After all elements have been checked, the pivot is placed in its correct position.
- The pivot is swapped with the element at index i+1:

arr[i+1], arr[high] = arr[high], arr[i+1]

- After this operation:
  - Elements smaller than the pivot are on the **left**.
  - The pivot is in its **correct position**.
  - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

return i+1

Condition 4: if low < high — Continue Quick Sort

- The function quick_sort(arr, low, high) is used to sort the array.
- The condition low < high checks whether the current part of the array contains more than one element.
- If the condition is **True**, the array needs to be sorted.
- The partition() function is called:

pi = partition(arr, low, high)

- pi stores the final position of the pivot.

Condition 5: quick_sort(arr, low, pi-1) — Sort Left Part

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, low, pi-1)

- The pivot itself is not included because it is already in its correct position.

Condition 6: quick_sort(arr, pi+1, high) — Sort Right Part

- The elements on the right side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, pi+1, high)

- Again, the pivot is not included because it is already correctly positioned.

Input Section

- n = int(input("enter the no of elements:")) takes the number of elements from the user.
- arr = [] creates an empty list to store the elements.
- The for loop runs n times and takes each element from the user:
- for i in range(n):

arr.append(int(input("enter the elements:")))

- Each entered value is converted into an integer and added to the array using append().

Calling the Quick Sort Function

quick_sort(arr, 0, n-1)

- This calls the quick_sort() function.
- 0 represents the first index of the array.
- n-1 represents the last index of the array. .

Error table:

sno	Error obtained	Rectification
1	IndexError	Check i, j, and pivot boundaries
2	Wrong swapping conditions	By shifting to the correct spaces

WEEK-4:

4A)AIM: write a python program to implement a single linked list which could perform the below operations by taking an input from the user.

Code:

cll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-P

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for _ in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.tail.next = new_node
            self.tail = new_node
```

cll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PY1

File Edit Format Run Options Window Help

```
    # iv. Insert at specific index
    def insert_at_index(self, value, index):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_beginning(value)
            return

        if self.head is None:
            print("Index out of range")
            return

        current = self.head
        position = 0

        while position < index - 1 and current != self.tail:
            current = current.next
            position += 1

        if position != index - 1:
            print("Index out of range")
            return

        new_node = Node(value)
        new_node.next = current.next
        current.next = new_node

        if current == self.tail:
            self.tail = new_node

    # v. Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        current = self.head
        previous = self.tail

        while True:
            if current.data == value:
                break

            previous = current
```

cll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.S

File Edit Format Run Options Window Help

```
            self.tail = None
            return

        # Delete head
        if current == self.head:
            self.head = self.head.next
            self.tail.next = self.head
            return

        # Delete other node
        previous.next = current.next

        # Delete tail
        if current == self.tail:
            self.tail = previous

    # vi. Delete at beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head == self.tail:
            self.head = None
            self.tail = None
        else:
            self.head = self.head.next
            self.tail.next = self.head

    # vii. Delete at end
    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head == self.tail:
            self.head = None
            self.tail = None
            return

        current = self.head

        while current.next != self.tail:
            current = current.next
```

cll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA

File Edit Format Run Options Window Help

```
        if self.head is None:
            return 0

        count = 0
        current = self.head

        while True:
            count += 1
            current = current.next

            if current == self.head:
                break

        return count

    # ix. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while True:
            print(current.data, end=" -> ")
            current = current.next

            if current == self.head:
                break

        print("(Head)")

    # x. Display head and tail
    def display_head_tail(self):
        if self.head is None:
            print("List is empty")
            return

        print("Head:", self.head.data)
        print("Tail:", self.tail.data)
        print("Tail points to:", self.tail.next.data)


cll = CircularLinkedList()
```

```
while True:

    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Display Head and Tail")
    print("11. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()

    elif choice == 2:
        value = int(input("Enter value: "))
        cll.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        cll.insert_end(value)

    elif choice == 4:
        value = int(input("Enter value: "))
        index = int(input("Enter index: "))
        cll.insert_at_index(value, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        cll.delete_by_value(value)

    elif choice == 6:
        cll.delete_beginning()

    elif choice == 7:
        cll.delete_end()

    elif choice == 8:
        print("Number of nodes:", cll.count_nodes())
```

Output:

```
========== RESTART: C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\D

----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 1
Enter number of nodes: 2
Enter value: 3
Enter value: 4

----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice:
```

Explanation:

1. A **singly linked list** is a collection of nodes where each node contains **two parts**: data and a link to the next node.
2. Each node contains:
   - **Data** – stores the value.
   - **Next (next)** – stores the reference/address of the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are created one by one and added to the list.
7. For the first node, both **head and tail** point to the same node, and the next of the tail is None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node's next points to the current head, and the new node becomes the new head.

9. During **insertion at the end**, the new node is added after the current tail. The current tail's next points to the new node, and the new node becomes the new tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the appropriate nodes by adjusting their next links.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by making the previous node point to the node after the deleted node.
12. If the node to be deleted is the **head**, the next node becomes the new head.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
16. During **deletion at the end**, the list is traversed to find the node before the tail. That node becomes the new tail and its next is set to None.
17. The **count operation** traverses the list from the head to the end and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves through the list using the next link until None is reached.
20. The program continues to execute until the user selects the **Exit** option.

Error table:

sno	Error obtained	rectification
1	Infinite loop or incomplete traversal due to not updating current	Move to the next node using temp = temp.next
2	Forgetting to update head while inserting/deleting at the beginning	Update head to point to the new first node using head = new_node or head = head.next

AIM: write a python program to implement a double linked list which could perform the below operations by taking an input from the user.

Code:

dll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for _ in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            return

        current = self.head

        while current.next:
            current = current.next

        current.next = new_node
        new_node.prev = current

    # iv. Insert at specific index
```

dll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE251

File Edit Format Run Options Window Help

```
    # iv. Insert at specific index
    def insert_at_index(self, value, index):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_beginning(value)
            return

        current = self.head

        for _ in range(index - 1):
            if current is None:
                print("Index out of range")
                return
            current = current.next

        if current is None:
            print("Index out of range")
            return

        new_node = Node(value)

        new_node.next = current.next
        new_node.prev = current

        if current.next is not None:
            current.next.prev = new_node

        current.next = new_node

    # v. Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current and current.data != value:
            current = current.next

        if current is None:
            print("Value not found")
            return
```

dll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25

File Edit Format Run Options Window Help

```
            return

        if current.prev is not None:
            current.prev.next = current.next
        else:
            self.head = current.next

        if current.next is not None:
            current.next.prev = current.prev

    # vi. Delete at beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

    # vii. Delete at end
    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current.next:
            current = current.next

        if current.prev is None:
            self.head = None
        else:
            current.prev.next = None

    # viii. Count nodes
    def count_nodes(self):
        count = 0
        current = self.head

        while current:
            count += 1
            current = current.next
```

dll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CS

File Edit Format Run Options Window Help

```
while True:

    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        value = int(input("Enter value: "))
        dll.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        dll.insert_end(value)

    elif choice == 4:
        value = int(input("Enter value: "))
        index = int(input("Enter index: "))
        dll.insert_at_index(value, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        dll.delete_by_value(value)

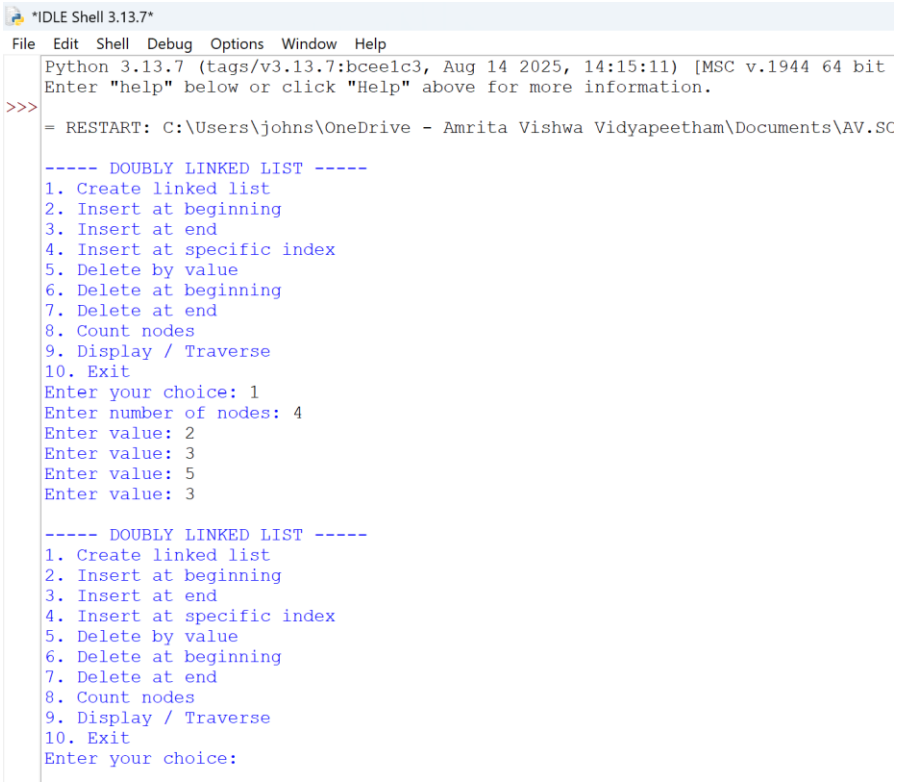
    elif choice == 6:
        dll.delete_beginning()

    elif choice == 7:
        dll.delete_end()

    elif choice == 8:
        print("Number of nodes:", dll.count_nodes())

    elif choice == 9:
```

Output:



Explanation:

1. A **doubly linked list** is a collection of nodes where each node contains **three parts**: data, a link to the previous node, and a link to the next node.
2. Each node has:
   - **Data** – stores the value.
   - **Previous (prev)** – points to the previous node.
   - **Next (next)** – points to the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and maintain the data, prev, and next references.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are added one by one at the end of the list.

7. For the first node, both **head and tail** point to the same node. The prev of the head and the next of the tail are None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node becomes the head, its next points to the previous head, and the previous head's prev points back to the new node.
9. During **insertion at the end**, the new node is placed after the current tail. The new node's prev points to the old tail, the old tail's next points to the new node, and the new node becomes the tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the previous and next nodes, and both prev and next links are updated.
11. During **deletion by value**, the list is searched for the node containing the given value. Once found, its previous and next nodes are connected to each other, removing the selected node from the list.
12. If the node to be deleted is the **head**, the next node becomes the new head and its prev is set to None.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head. The new head's prev is set to None.
16. During **deletion at the end**, the tail node is removed and the previous node becomes the new tail. The new tail's next is set to None.
17. The **count operation** traverses the list from the head to the tail and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves forward using the next link to display all the nodes.
19. Since it is a doubly linked list, the program can also traverse the list **backwards**, starting from the tail and using the prev links.
20. The program displays the **head and tail** values to show the starting and ending nodes of the list.
21. The important relationships maintained in a doubly linked list are:

head.prev = None

tail.next = None

22. The overall structure of a doubly linked list is:

None ← [10] ⇄ [20] ⇄ [30] ⇄ [40] → None

↑ ↑

HEAD TAIL

23. The arrows in both directions allow the list to be traversed **forward as well as backward**.

24. The program is **menu-driven**, allowing the user to perform creation, insertion, deletion, counting, displaying, and other operations repeatedly until the **Exit** option is selected.

Error table:

Sno	Error obtained	Rectification
1	Forgetting to update the prev pointer	Update both prev and next pointers whenever a node is inserted
2	Incorrect head.prev	Maintain head.prev = None

AIM: write a python program to implement a circular linked list which could perform the below operations by taking an input from the user

Code:

sll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)
        new_node.next = self.head
        self.head = new_node

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            return

        current = self.head

        while current.next:
            current = current.next

        current.next = new_node

    # iv. Insert at specific index
    def insert_at_index(self, value, index):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
```

sll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115

File Edit Format Run Options Window Help

```
        for _ in range(index - 1):
            if current is None:
                print("Index out of range")
                return
            current = current.next

        if current is None:
            print("Index out of range")
            return

        new_node = Node(value)
        new_node.next = current.next
        current.next = new_node

    # v. Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        if self.head.data == value:
            self.head = self.head.next
            return

        current = self.head

        while current.next and current.next.data != value:
            current = current.next

        if current.next is None:
            print("Value not found")
        else:
            current.next = current.next.next

    # vi. Delete at beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

    # vii. Delete at end
    def delete_end(self):
        if self.head is None:
```

sll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-

File Edit Format Run Options Window Help

```
        if self.head is None:
            print("List is empty")
            return

        if self.head.next is None:
            self.head = None
            return

        current = self.head

        while current.next.next:
            current = current.next

        current.next = None

    # viii. Count number of nodes
    def count_nodes(self):
        count = 0
        current = self.head

        while current:
            count += 1
            current = current.next

        return count

    # ix. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current:
            print(current.data, end=" -> ")
            current = current.next

        print("None")


linked_list = LinkedList()

while True:

    print("\n----- SINGLY LINKED LIST -----")
```

sll.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTHC

File Edit Format Run Options Window Help

```
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        linked_list.create()

    elif choice == 2:
        value = int(input("Enter value: "))
        linked_list.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        linked_list.insert_end(value)

    elif choice == 4:
        value = int(input("Enter value: "))
        index = int(input("Enter index: "))
        linked_list.insert_at_index(value, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        linked_list.delete_by_value(value)

    elif choice == 6:
        linked_list.delete_beginning()

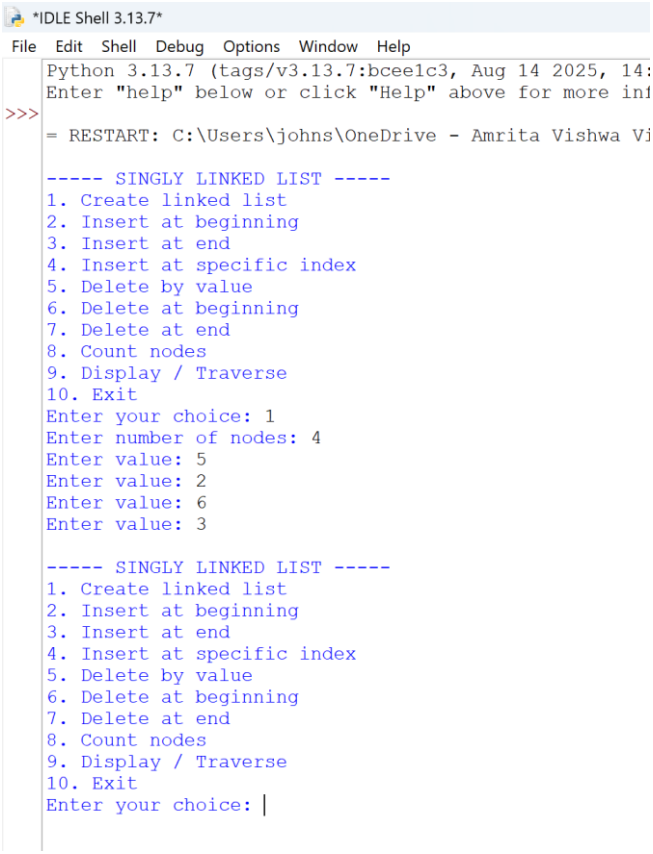
    elif choice == 7:
        linked_list.delete_end()

    elif choice == 8:
        print("Number of nodes:", linked_list.count_nodes())

    elif choice == 9:
        linked_list.display()

    elif choice == 10:
        print("Program exited.")
```

Output:



Explanation :

1. A **circular singly linked list** is a collection of nodes where each node contains **data** and a **link to the next node**.
2. In a circular linked list, the **last node points back to the first node (head)** instead of pointing to None.
3. The program uses two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. Each new node is inserted at the end of the list.
7. If the list is empty, the first node becomes both the **head and tail**, and its next points back to itself.

8. During **insertion at the beginning**, the new node is placed before the current head. The new node becomes the head, and the tail is connected to the new head.
9. During **insertion at the end**, the new node is added after the current tail. The new node becomes the tail, and its next points to the head.
10. During **insertion at a specific index**, the list is traversed until the required position is reached, and the new node is inserted between two existing nodes.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by connecting its previous node directly to its next node.
12. If the node to be deleted is the **head**, the next node becomes the new head and the tail is connected to it.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is connected to the head.
14. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
15. During **deletion at the end**, the node before the tail is found and made the new tail.
16. The **count operation** traverses the list and counts all the nodes. Traversal stops when the head is reached again.
17. The **display/traverse operation** starts from the head and visits every node until it reaches the head again.
18. Since the list is circular, traversal does not use current != None. Instead, it stops when current becomes equal to head.
19. The program also displays the **head, tail, and tail.next** to show the circular connection.
20. The program is **menu-driven**, so the user can repeatedly perform different operations until the **Exit** option is selected.
21. The most important property maintained throughout the program is:

tail.next = head

22. Thus, the structure of the circular linked list is:

HEAD → Node → Node → Node → TAIL

↑ |

|_____|

This ensures that the linked list remains **circular after every insertion and deletion operation**.

Error table:

sno	Error obtained	Rectification
1	Using temp != None as the traversal condition	Stop traversal when temp == head
2	Forgetting to update tail.next when the head changes	After changing the head, update tail.next = head

WEEK-5:

5A)Aim: write a python program to implement the stack using the arrays. And also implement stack operations such as push, pop, peek and also display the stack elements.

Code:

1.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DSA-PYTH

File Edit Format Run Options Window Help

```
class Stack:
    def __init__(self, size):
        self.stack = []
        self.size = size

    # Push operation
    def push(self, value):
        if len(self.stack) == self.size:
            print("Stack Overflow")
        else:
            self.stack.append(value)
            print(value, "pushed into stack")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow")
        else:
            value = self.stack.pop()
            print("Popped element:", value)

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[-1])

    # Display operation
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:")

            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])


size = int(input("Enter stack size: "))
s = Stack(size)
```

```
while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        s.push(value)

    elif choice == 2:
        s.pop()

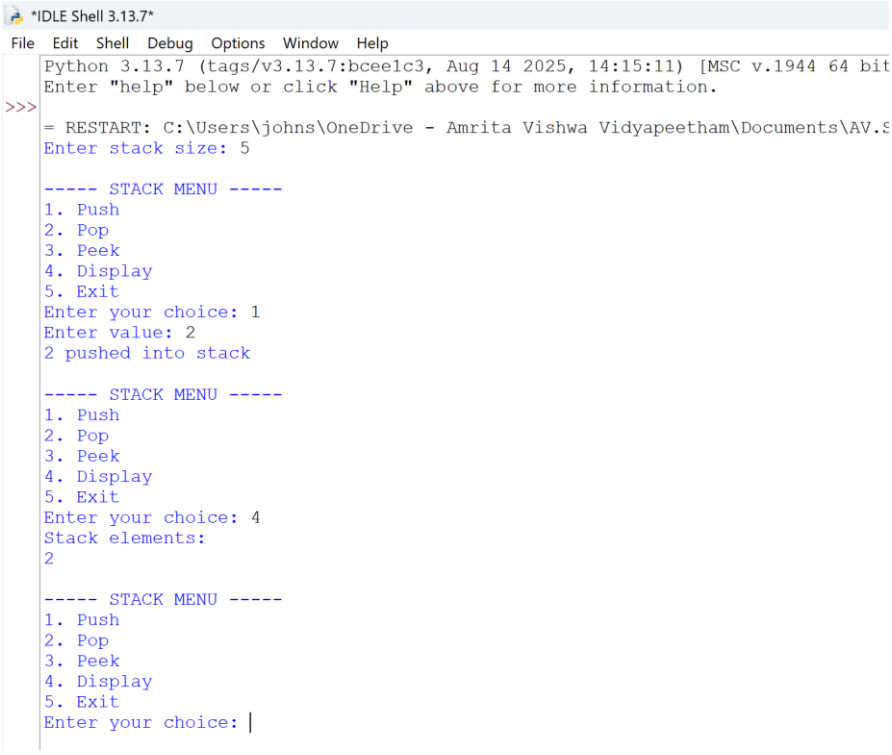
    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Class and Initialization

- A class named Stack is created.
- The __init__() method acts as the constructor.
- An empty list self.stack is created to store stack elements.
- The stack follows the **LIFO (Last In, First Out)** principle.

2. Push Operation

- The push() method is used to insert an element into the stack.
- The append() function adds the element to the end of the list.
- The newly inserted element becomes the **top element**.
- A message is displayed after the element is inserted.

3. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether the stack is empty.
- If the stack is empty, **Stack Underflow** is displayed.
- Otherwise, the pop() function removes the last element.

- The removed element is displayed.

4. Peek Operation

- The peek() method is used to view the top element.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.stack[-1] is used to access the top element.
- The top element is displayed without removing it.

5. Display Operation

- The display() method is used to display all stack elements.
- The program first checks whether the stack is empty.
- If empty, **Stack is empty** is displayed.
- Otherwise, a for loop is used to traverse the stack.
- The elements are displayed from **top to bottom**.

6. Count Operation

- The count() method is used to find the number of elements in the stack.
- The len() function returns the number of elements.
- The count is displayed to the user.

Error table:

sno	Error obtained	Error rectification
1	Stack underflow	Check whether the stack is empty before poping
2	Name error	Define the class before creating the object

5B)Aim: write a python program to implement the stack using the linked list. And also implement stack operations such as push, pop, peek and also display the stack elements.

Code:

2.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE25115\DS/

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None
        self.count = 0

    # Push operation
    def push(self, value):
        new_node = Node(value)

        new_node.next = self.top
        self.top = new_node
        self.count += 1

        print(value, "pushed into stack")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
            return

        value = self.top.data
        self.top = self.top.next
        self.count -= 1

        print("Popped element:", value)

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Count operation
    def count_nodes(self):
        print("Number of elements:", self.count)

    # Display operation
    def display(self):
```

1.py - C:\Users\johns\OneDrive - Amrita Vishwa Vidyapeetham\Documents\AV.SC.U4CSE251

File Edit Format Run Options Window Help

```
            print("Top element:", self.stack[-1])

    # Display operation
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:")

            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])


size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        s.push(value)

    elif choice == 2:
        s.pop()

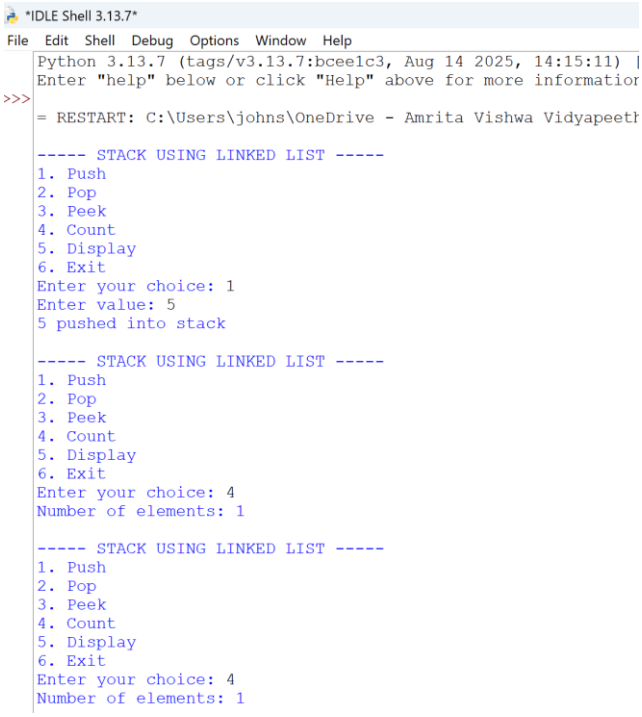
    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Node Class

- A class named Node is created to represent each element of the linked list.
- The __init__() method initializes a node.
- self.data stores the value of the node.
- self.next stores the reference to the next node.
- Initially, self.next is set to None.

2. Stack Class

- A class named Stack is created to implement the stack operations.
- The __init__() method initializes the stack.
- self.top = None indicates that the stack is initially empty.
- The top variable stores the reference to the top node of the stack.

3. Push Operation

- The push() method is used to insert an element into the stack.
- A new node is created using Node(data).
- The new node is connected to the current top node using:

new_node.next = self.top

- The new node is then made the top node using:

self.top = new_node

- The inserted element is displayed.
- The operation follows the **LIFO (Last In, First Out)** principle.

4. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether self.top is None.
- If self.top is None, the stack is empty and **Stack Underflow** is displayed.
- Otherwise, the data of the top node is stored in data.
- The top is moved to the next node using:

self.top = self.top.next

- The removed element is displayed.

5. Peek Operation

- The peek() method is used to view the top element without removing it.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.top.data is used to display the top element.
- The stack remains unchanged after the peek operation.

6. Display Operation

- The display() method is used to display all the elements of the stack.
- The program first checks whether the stack is empty.
- A temporary variable current is initialized with self.top.
- A while loop is used to traverse the linked list.
- current.data displays the value of each node.
- current = current.next moves to the next node.
- The elements are displayed from **top to bottom**.

7. Count Operation

- The count() method is used to find the number of elements in the stack.
- A variable count is initialized to 0.
- A temporary variable current starts from self.top.
- The linked list is traversed using a while loop.
- The count value is increased for every node.
- The total number of elements is displayed.

Error table:

sno	Error obtained	Error rectification
1	Incorrect push	Set new_node.next = self.top before updating self.top
2	Infinte loop	Use break when choice == 6

WEEK-6:

_6) AIM:Write a program to implement stack operations such as push,pop,peek,display using an array and a linked list.

Code:



```
6A.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-6/6A.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
        value = int(input("Enter element: "))
        stack.append(value)
        print("Element pushed")

def pop():
    if len(stack) == 0:
        print("Stack Underflow")
    else:
        print("Popped element:", stack.pop())

def peek():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Top element:", stack[-1])

def display():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Stack elements:")
        for i in range(len(stack) - 1, -1, -1):
            print(stack[i])

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

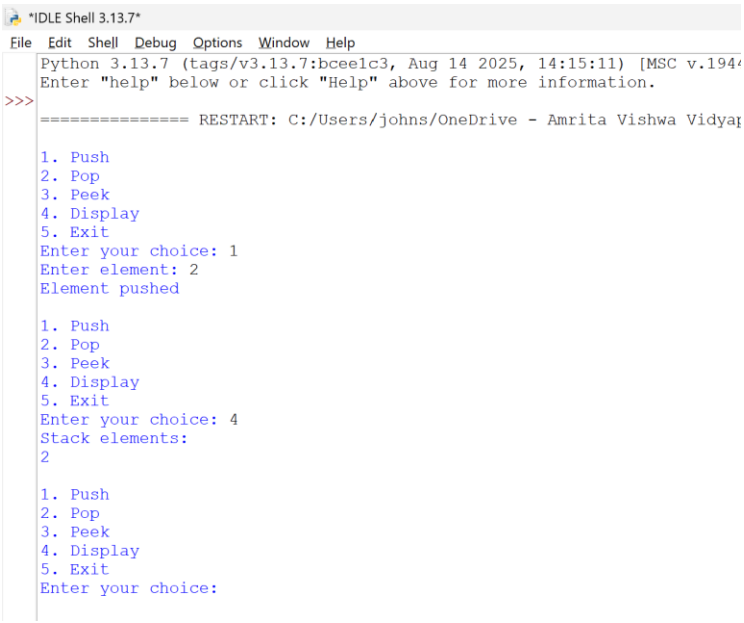
```
            print(stack[i])

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

 Stack = [] and MAX = 5: Initializes an empty list named stack to store elements and sets the maximum size limit to 5.

 push() Function: Checks if len(stack) == MAX. If full, it displays "Stack Overflow"; otherwise, it takes an integer input, appends it to stack, and confirms addition.

 pop() Function: Checks if len(stack) == 0. If empty, it displays "Stack Underflow"; otherwise, it removes and prints the top element using stack.pop().

 peek() Function: Checks if the stack is empty. If not, it accesses and displays the top element (stack[-1]) without removing it.

 display() Function: Iterates backward from len(stack) - 1 down to 0 using range() to display elements from top to bottom.

 Main Menu Loop (while True): Continually prompts the user with choices (1-5) to execute stack operations until option 5 is selected to terminate the program.

Error Table:

sno	Error obtained	Error rectification
1	Incorrect stack overflow or underflow check	Ensure proper checking using len(stack) == MAX for push and len(stack) == 0 for pop/peek
2	ValueError on non-integer input	Wrap int(input(...)) calls in a try-except ValueError block to handle invalid inputs safely
3	Infinite loop on menu exit	Ensure break is executed when choice == 5 inside the loop condition

Stack implementation using Linked List

Code:

6B.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-6/6B.py (3.

File Edit Format Run Options Window Help

```
# Stack implementation using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    def push(self):
        value = int(input("Enter element: "))

        new_node = Node(value)
        new_node.next = self.top
        self.top = new_node

        print("Element pushed")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top

            print("Stack elements:")
            while temp:
                print(temp.data)
                temp = temp.next
```

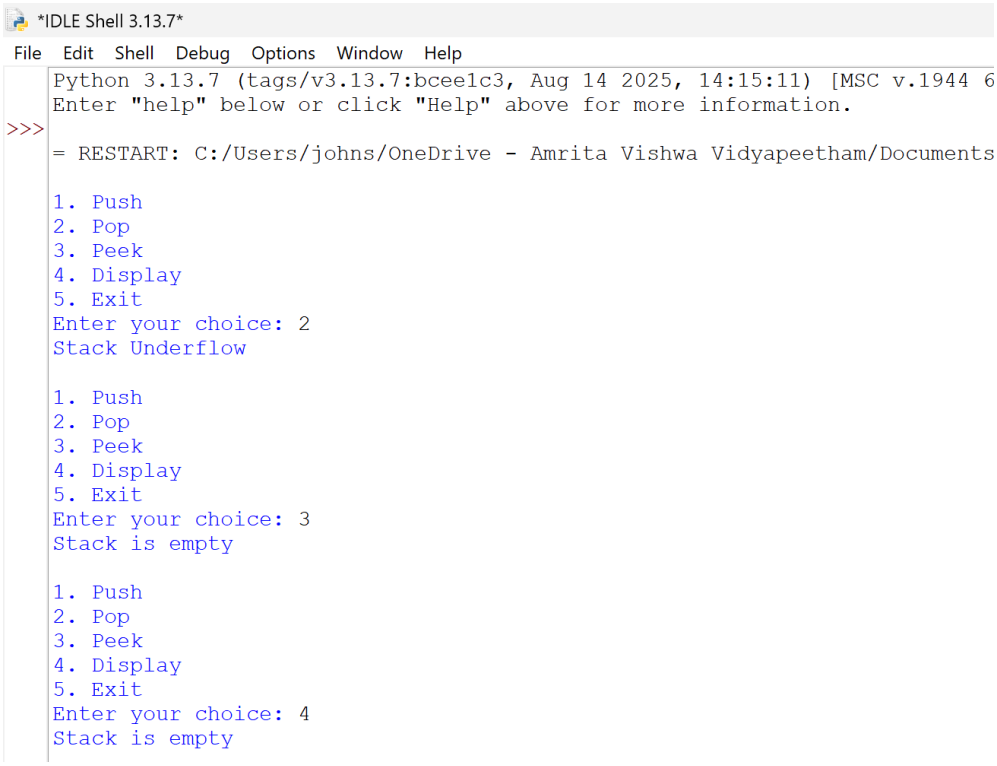
```
s = Stack()

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        s.push()
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

 Node Class: Represents each element in the stack with two properties: data (stores the element value) and next (pointer/reference to the next node in the list).

 Stack Class Initialization: Initializes self.top = None to signify that the stack starts empty.

 push() Method: Accepts user input, wraps it in a new Node, links new_node.next to the current self.top, and updates self.top to point to new_node.

 pop() Method: Checks if self.top is None (Stack Underflow). If not empty, it prints self.top.data and shifts self.top to self.top.next to remove the top node.

 peek() Method: Checks if the stack is empty. If nodes exist, it displays the data stored in the top node (self.top.data) without modifying the stack.

 display() Method: Traverses the linked list from self.top down to the end using a temporary pointer temp, printing each temp.data until temp becomes None.

 Main Menu Loop (while True): Instantiates s = Stack() and continuously prompts the user to select operations (1–5), invoking the corresponding class method until choice == 5 breaks the loop.

Error table:

sno	Error obtained	Error rectification
1	Incorrect push sequence	Set new_node.next = self.top before updating self.top = new_node
2	Infinite loop on display	Update traversal pointer using temp = temp.next inside the while temp: loop
3	Program crash on non-integer input	Wrap int(input(...)) calls in a try-except ValueError block

Week-7:

7A) AIM: write a python program to implement queue and its operations such as enqueue, dequeue, peek,display with a menu driven approach implement using array, and linked list data structures.

Code:

7A1.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-7/7A1.py (3.13.7)

File Edit Format Run Options Window Help

```
# Queue implementation using Array

queue = []
MAX = 5

def enqueue():
    if len(queue) == MAX:
        print("Queue Overflow")
    else:
        value = int(input("Enter element: "))
        queue.append(value)
        print("Element inserted")


def dequeue():
    if len(queue) == 0:
        print("Queue Underflow")
    else:
        print("Deleted element:", queue.pop(0))


def peek():
    if len(queue) == 0:
        print("Queue is empty")
    else:
        print("Front element:", queue[0])

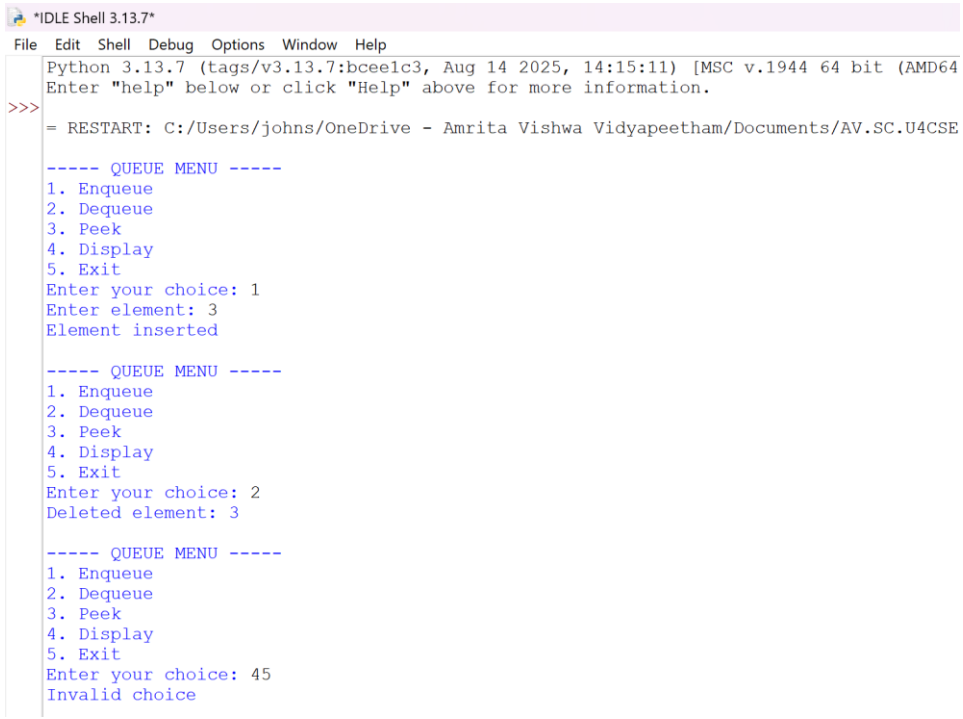

def display():
    if len(queue) == 0:
        print("Queue is empty")
    else:
        print("Queue elements:")
        for element in queue:
            print(element, end=" ")
        print()


while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

Output:



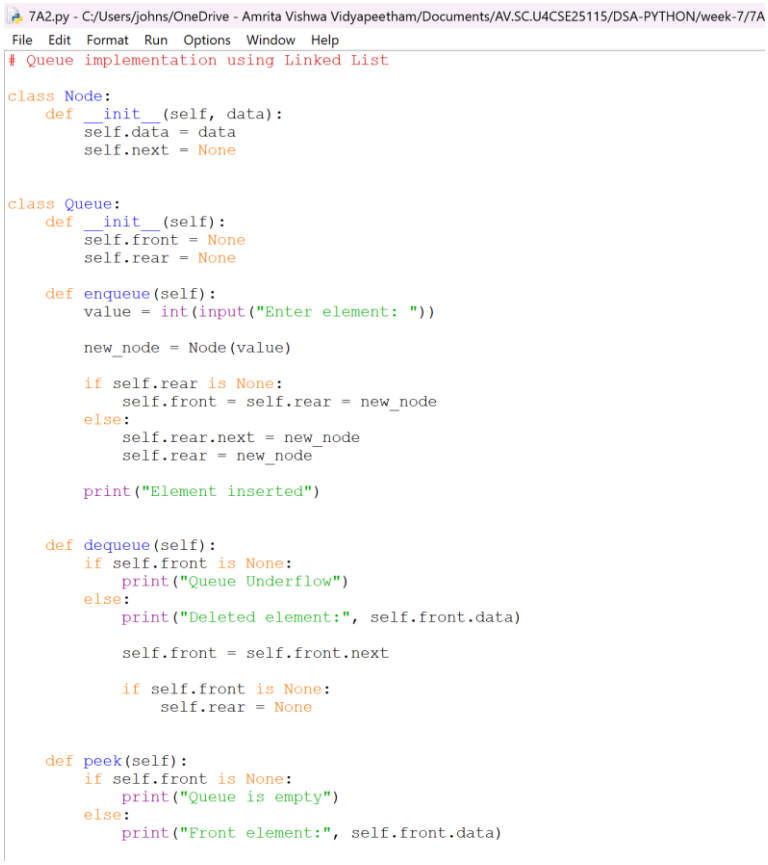
```
*IDLE Shell 3.13.7*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 3
Element inserted

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 3

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 45
Invalid choice
```

Queue implementation using Linked List



```
7A2.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-7/7A
File  Edit  Format  Run  Options  Window  Help
# Queue implementation using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self):
        value = int(input("Enter element: "))

        new_node = Node(value)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print("Element inserted")


    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            print("Deleted element:", self.front.data)

            self.front = self.front.next

            if self.front is None:
                self.rear = None


    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)
```

7A2.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHOI

File Edit Format Run Options Window Help

```
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)


    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            temp = self.front

            print("Queue elements:")
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()


q = Queue()

while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        q.enqueue()
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 b
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Queue Underflow

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Queue Underflow

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty
```

Explanation:

- **Array-Based Implementation**:
  - **queue = [] & MAX = 5**: Initializes an empty list with a maximum capacity of 5.
  - **enqueue()**: Checks if len(queue) == MAX to prevent overflow, then prompts for an integer and adds it to the back using queue.append().
  - **dequeue()**: Checks if len(queue) == 0 (underflow). If not empty, it removes and prints the first element using queue.pop(0).
  - **peek()**: Displays the front element (queue[0]) without removing it.
  - **display()**: Iterates through the list sequentially from front to rear and prints the elements on a single line.
- **Linked List-Based Implementation**:
  - **Node Class**: Stores the data payload and a next reference to the subsequent node.
  - **Queue Class**: Tracks two pointers: self.front (where elements are removed) and self.rear (where elements are added). Both start as None.
  - **enqueue()**: Creates a new node. If the queue is empty (self.rear is None), both front and rear point to new_node. Otherwise, it links self.rear.next to new_node and updates self.rear.
  - **dequeue()**: Removes the node at self.front by advancing self.front = self.front.next. If the queue becomes empty as a result, self.rear is also reset to None.
  - **peek() & display()**: peek() displays self.front.data, while display() uses a temporary pointer temp to traverse from front to rear, printing each node's value.
- **Menu Loop**:
  - Both implementations utilize an interactive while True loop providing a menu choices (1–5) to trigger operations until option 5 executes a break.

Error Table:

sno	Error obtained	Error rectification
1	Incorrect enqueue sequence in linked list	Set self.rear.next = new_node before updating self.rear = new_node
2	Dangling rear pointer on last dequeue	Reset self.rear = None when self.front becomes None after a dequeue
3	Program crash on invalid input	Wrap int(input(...)) calls in a try-except ValueError block

Week-8:

A)AIM:Implement Binary Tree with operations using Array.

Code:




```
# Main Program
bt = BinaryTree(10)

bt.insert_root(10)
bt.insert_left(0, 20)
bt.insert_right(0, 30)
bt.insert_left(1, 40)
bt.insert_right(1, 50)

bt.display()
bt.search(30)
bt.delete(20)
bt.display()
```

Output:

```
= RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-8/A.py
Binary Tree: [10, 20, 30, 40, 50, None, None, None, None, None]
Element found
Element deleted
Binary Tree: [10, None, 30, 40, 50, None, None, None, None, None]
```

Explanation:

Operation	Purpose
Insert -	Adds a node
Display -	Prints the array
Search -	Finds an element
Delete -	Removes an element

A Binary Tree is a non-linear data structure in which each node can have a maximum of two children:

1. Left child
2. Right child

In this program, we implement a Binary Tree using an array instead of linked nodes.

Error table

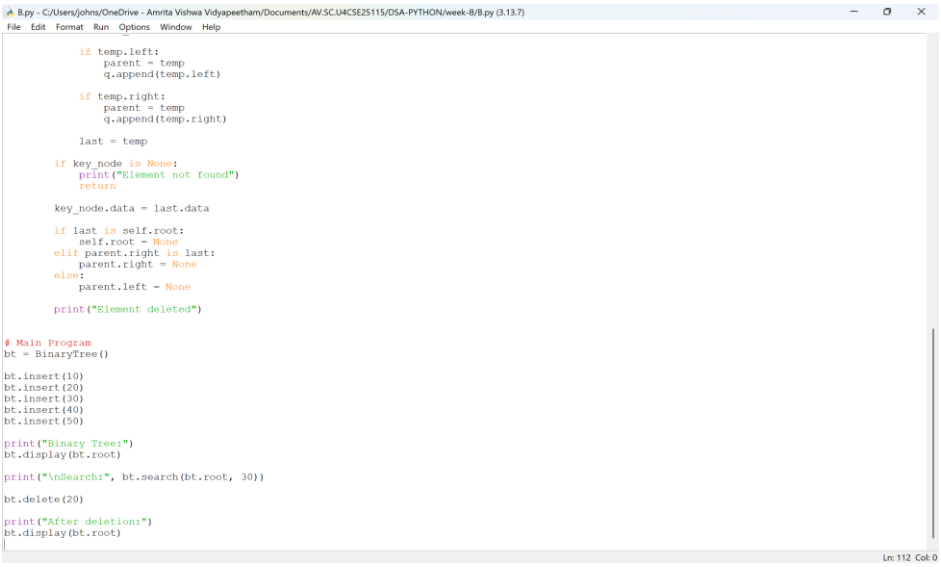
These are common errors you may encounter while writing this program in the lab exam.

Error / Mistake	Reason	Correction
IndexError	Accessing an invalid array index	Check i < self.size
AttributeError	Incorrect object or method name	Use bt.display()
TypeError	Passing incorrect arguments	Pass the required values
IndentationError	Incorrect spacing	Maintain proper indentation
SyntaxError	Missing colon :	Add : after function and class definitions

NameError	Using an undefined variable	Define the variable first
Element not found	Value does not exist	Check the inserted value

B)Implement Binary Tree with operations using Linked List

Code:



```
            if temp.left:
                parent = temp
                q.append(temp.left)

            if temp.right:
                parent = temp
                q.append(temp.right)

            last = temp

        if key_node is None:
            print("Element not found")
            return

        key_node.data = last.data

        if last is self.root:
            self.root = None
        elif parent.right is last:
            parent.right = None
        else:
            parent.left = None

        print("Element deleted")

# Main Program
bt = BinaryTree()

bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(40)
bt.insert(50)

print("Binary Tree:")
bt.display(bt.root)

print("\nSearch:", bt.search(bt.root, 30))

bt.delete(20)

print("After deletion:")
bt.display(bt.root)
```

Output:

```
= RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-8/B.py
Binary Tree:
10 20 40 50 30
Search: True
Element deleted
After deletion:
10 50 40 30
```

Explanation:

A Binary Tree is a non-linear data structure in which each node contains at most two children: a left child and a right child.

In Linked List implementation, each node consists of three parts:

1. Data: Stores the value of the node.
2. Left: Stores the reference to the left child.
3. Right: Stores the reference to the right child.

The root is the first node of the Binary Tree.

2. Explanation of the Program

1. Node Creation

- A Node class is created to represent each node in the Binary Tree.
- The constructor initializes the node with data.
- Each node contains data, left and right references.
- Initially, left and right references are set to None.

2. Binary Tree Initialization

- A BinaryTree class is created.
- The root is initialized to None.
- None indicates that the tree is initially empty.

3. Insertion Operation

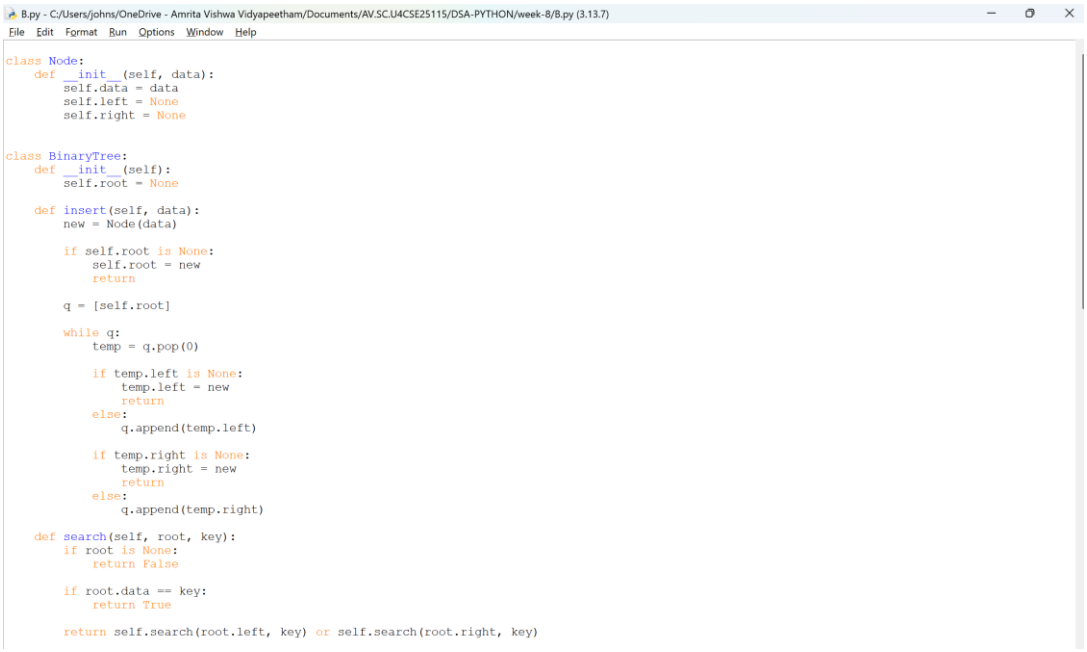
- A new node is created with the given value.
- If the tree is empty, the new node becomes the root.
- Otherwise, a queue is created to insert elements level by level.

Error	Reason	Solution
AttributeError	Accessing an invalid node attribute	Check left and right references
NameError	Using an undefined variable	Declare the variable
TypeError	Passing incorrect arguments	Pass correct arguments
IndentationError	Incorrect indentation	Maintain proper spacing
RecursionError	Excessive recursive calls	Check recursion conditions

Week-9:

1)AIM:Python program to implement Binary Search Tree (BST) operations

Code:



```
B.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-8/B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new = Node(data)

        if self.root is None:
            self.root = new
            return

        q = [self.root]

        while q:
            temp = q.pop(0)

            if temp.left is None:
                temp.left = new
                return
            else:
                q.append(temp.left)

            if temp.right is None:
                temp.right = new
                return
            else:
                q.append(temp.right)

    def search(self, root, key):
        if root is None:
            return False

        if root.data == key:
            return True

        return self.search(root.left, key) or self.search(root.right, key)
```



```
B.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-8/B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
        if root.data == key:
            return True

        return self.search(root.left, key) or self.search(root.right, key)

    def display(self, root):
        if root:
            print(root.data, end=" ")
            self.display(root.left)
            self.display(root.right)

    def delete(self, key):
        if self.root is None:
            print("Tree is empty")
            return

        q = [self.root]
        key_node = None
        last = None
        parent = None

        while q:
            temp = q.pop(0)

            if temp.data == key and key_node is None:
                key_node = temp

            if temp.left:
                parent = temp
                q.append(temp.left)

            if temp.right:
                parent = temp
                q.append(temp.right)

            last = temp

        if key_node is None:
            print("Element not found")
            return

        key_node.data = last.data

        if last is self.root:
            self.root = None
        elif parent.right is last:
```

Output:

```
= RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-9/BST.py
BST Elements:
20 30 40 50 60 70 80
Search: Found
After Deletion:
20 40 50 60 70 80
```

Node Creation

- A Node class is created to represent each node in the BST.
- Each node contains three parts: data, left and right.
- Data stores the value of the node.
- Left and right store references to the child nodes.
- Initially, both child references are set to None.

2. BST Initialization

- A BST class is created to perform tree operations.
- The root is initialized to None.
- Root represents the first node of the tree.
- Initially, the tree is empty.

3. Search Operation

- The search operation is used to find a particular element in the BST.
- First, it checks whether the tree is empty or the required element is found.
- If the search value is smaller than the current node, it searches the left subtree.
- If the search value is greater, it searches the right subtree.
- If the element is found, it returns the node.
- If the element is not found, it returns None.

4. Inorder Traversal

- Inorder traversal is used to display the elements of the BST in sorted order.
- First, it visits the left subtree.
- Next, it visits the root node.

- Finally, it visits the right subtree.
- The traversal follows Left → Root → Right order.

6. Deletion Operation

- The deletion operation is used to remove an element from the BST.
- First, the required element is searched.
- If the node has no children, it is directly removed.
- If the node has only one child, it is replaced by its child.
- If the node has two children, the inorder successor is found.
- The inorder successor is the smallest value in the right subtree.
- The node's value is replaced with the inorder successor's value.
- Finally, the inorder successor node is deleted.

3. Error Table

Error	**Reason**	**Solution**
AttributeError	Accessing an invalid node attribute	Check node references
NameError	Using an undefined variable	Declare the variable
TypeError	Passing incorrect arguments	Pass correct arguments
IndentationError	Incorrect indentation	Maintain proper spacing
RecursionError	Excessive recursive calls	Check recursion conditions
SyntaxError	Missing colon or brackets	Correct the syntax
NoneType Error	Accessing data from an empty node	Check whether the node is None

Week-10:

!)AIM:Write a Python program to implement an AVL Tree with all its operations.

Code:

AVL trees.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-10/AVL trees.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:

    def height(self, root):
        if root is None:
            return 0
        return root.height

    def balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def right_rotate(self, y):
        x = y.left
        t2 = x.right

        x.right = y
        y.left = t2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    def left_rotate(self, x):
        y = x.right
        t2 = y.left

        y.left = x
        x.right = t2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
```

AVL trees.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-10/AVL trees.py (3.13.7)

File Edit Format Run Options Window Help

```
    def left_rotate(self, x):
        y = x.right
        t2 = y.left

        y.left = x
        x.right = t2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))

        return y

    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        bf = self.balance(root)

        # LL Case
        if bf > 1 and key < root.left.key:
            return self.right_rotate(root)

        # RR Case
        if bf < -1 and key > root.right.key:
            return self.left_rotate(root)

        # LR Case
        if bf > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if bf < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
```

AVL trees.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-10/AVL trees.py (3.13.7)

File Edit Format Run Options Window Help

```
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)

        elif key > root.key:
            root.right = self.delete(root.right, key)

        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.min_value(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        bf = self.balance(root)

        # LL Case
        if bf > 1 and self.balance(root.left) >= 0:
            return self.right_rotate(root)

        # LR Case
        if bf > 1 and self.balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if bf < -1 and self.balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if bf < -1 and self.balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
```

AVL trees.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-10/AVL trees.py (3.13.7)

File Edit Format Run Options Window Help

```
        return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")


# Main Program
avl = AVLTree()
root = None

values = [10, 20, 30, 40, 50, 25]

for i in values:
    root = avl.insert(root, i)

print("Inorder Traversal:")
avl.inorder(root)

print("\nPreorder Traversal:")
avl.preorder(root)

print("\nPostorder Traversal:")
avl.postorder(root)

key = 30
if avl.search(root, key):
    print("\nElement Found")
else:
    print("\nElement Not Found")
```

Output:

```
= RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-10/AVL trees.py
Inorder Traversal:
10 20 25 30 40 50
Preorder Traversal:
30 20 10 25 40 50
Postorder Traversal:
10 25 20 50 40 30
Element Found
After Deletion:
10 20 25 30 50
```

Explanation:

Explanation of the Program

1. Node Creation

- A Node class is created to represent each node in the AVL Tree.
- Each node contains three parts: data, left and right references.
- It also contains a height variable to store the height of the node.
- Initially, left and right are set to None.
- The height of a newly created node is 1.

2. AVL Tree Initialization

- An AVLTree class is created to perform tree operations.
- The height function returns the height of a particular node.
- If the node is empty, its height is considered 0.

3. Balance Factor Calculation

- The balance factor is used to check whether the tree is balanced.
- It is calculated by subtracting the height of the right subtree from the height of the left subtree.
- If the balance factor is -1, 0 or +1, the node is balanced.
- If the balance factor is greater than +1 or less than -1, the tree becomes unbalanced.
- Rotations are performed to restore balance.

4. Right Rotation

- Right rotation is performed when the tree becomes unbalanced in the Left-Left (LL) case.
- The left child becomes the new root of that subtree.

- The previous root becomes the right child.
- The heights of the affected nodes are updated.

5. Left Rotation

- Left rotation is performed when the tree becomes unbalanced in the Right-Right (RR) case.
- The right child becomes the new root of that subtree.
- The previous root becomes the left child.
- The heights of the affected nodes are updated.

6. Insertion Operation

- The insertion operation adds a new element to the AVL Tree.
- First, a new node is created with the given value.
- If the tree is empty, the new node becomes the root.
- Smaller values are inserted into the left subtree.
- Greater values are inserted into the right subtree.
- After insertion, the height and balance factor are updated.
- If the tree becomes unbalanced, the appropriate rotation is performed.

7. Four Rotation Cases

- LL Case: Occurs when a node is inserted into the left subtree of the left child. It requires a right rotation.
- RR Case: Occurs when a node is inserted into the right subtree of the right child. It requires a left rotation.
- LR Case: Occurs when a node is inserted into the right subtree of the left child. It requires a left rotation followed by a right rotation.
- RL Case: Occurs when a node is inserted into the left subtree of the right child. It requires a right rotation followed by a left rotation.

8. Minimum Value Operation

- This operation finds the smallest value in a subtree.
- It starts from the given node.
- It continuously moves towards the left child.
- The last leftmost node contains the minimum value.

- It is mainly used during deletion.

9. Deletion Operation

- The deletion operation removes a particular element from the AVL Tree.
- First, the required node is searched.
- If the node has no children, it is directly removed.
- If the node has one child, it is replaced by its child.
- If the node has two children, the inorder successor is found.
- The node's value is replaced with the inorder successor's value.
- After deletion, the height and balance factor are updated.
- If the tree becomes unbalanced, suitable rotations are performed.

3. Error Table

Error	**Reason**	**Solution**
AttributeError	Accessing an invalid node attribute	Check node references
NameError	Using an undefined variable	Declare the variable
TypeError	Passing incorrect arguments	Pass correct arguments
IndentationError	Incorrect indentation	Maintain proper spacing
RecursionError	Excessive recursive calls	Check recursion conditions
SyntaxError	Missing colon or brackets	Correct the syntax
NoneType Error	Accessing data from an empty node	Check whether the node is None
AttributeError: 'NoneType'	Performing rotation on a missing child	Check child availability before rotation

Week-11:

1)AIM:Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing.

Code:

hash table.py - C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-11/hash table.py (3.13.7)

File Edit Format Run Options Window Help

```
SIZE = 10

# Separate Chaining
class SeparateChaining:
    def __init__(self):
        self.table = [[] for i in range(SIZE)]

    def insert(self, key):
        index = key % SIZE
        self.table[index].append(key)

    def display(self):
        print("Separate Chaining:")
        for i in range(SIZE):
            print(i, ":", self.table[i])


# Linear Probing
class LinearProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            new_index = (index + i) % SIZE

            if self.table[new_index] is None:
                self.table[new_index] = key
                return

        print("Hash Table is Full")

    def display(self):
        print("Linear Probing:")
        print(self.table)


# Quadratic Probing
class QuadraticProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
```

```
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            new_index = (index + i * i) % SIZE

            if self.table[new_index] is None:
                self.table[new_index] = key
                return

        print("Hash Table is Full")

    def display(self):
        print("Quadratic Probing:")
        print(self.table)


# Main Program
values = [23, 43, 13, 27, 37]

sc = SeparateChaining()
lp = LinearProbing()
qp = QuadraticProbing()

for key in values:
    sc.insert(key)
    lp.insert(key)
    qp.insert(key)

sc.display()
lp.display()
qp.display()
```

Output:

```
========== RESTART: C:/Users/johns/OneDrive - Amrita Vishwa Vidyapeetham/Documents/AV.SC.U4CSE25115/DSA-PYTHON/week-11/hash table.py =======
Separate Chaining:
0 : []
1 : []
2 : []
3 : [23, 43, 13]
4 : []
5 : []
6 : []
7 : [27, 37]
8 : []
9 : []
Linear Probing:
[None, None, None, 23, 43, 13, None, 27, 37, None]
Quadratic Probing:
[None, 37, None, 23, 43, None, None, 13, 27, None]
```

Explanation:

Table Size Initialization

- The hash table size is initialized to 10.
- It determines the number of available positions in the table.
- The modulo operator is used to calculate valid indexes.

2. Separate Chaining

- A SeparateChaining class is created to handle collisions.
- An empty list is created at every index of the hash table.
- The hash function calculates the index using the key modulo table size.
- The given key is inserted into the corresponding list.
- If multiple keys produce the same index, they are stored in the same list.
- The display operation prints all indexes and their corresponding elements.

3. Linear Probing

- A LinearProbing class is created to handle collisions.
- An empty hash table is initialized with None values.
- The hash function calculates the original index.
- If the calculated position is empty, the element is inserted.

- If the position is already occupied, the next position is checked.
- This process continues until an empty position is found.
- The modulo operator helps return to the beginning of the table when the end is reached.
- If no empty position is available, the program displays a Hash Table Full message.

4. Quadratic Probing

- A QuadraticProbing class is created to handle collisions.
- An empty hash table is initialized.
- The hash function calculates the original index.
- If the calculated position is occupied, another position is calculated using the square of the attempt number.
- The next available position is checked using quadratic increments.
- This process continues until an empty position is found.
- If no suitable position is found within the attempts, the program displays a Hash Table Full message.

5. Main Program

- A list of five integer values is created.
- Three objects are created for Separate Chaining, Linear Probing and Quadratic Probing.
- Each value is inserted into all three hash tables.
- The insertion process handles collisions using the respective techniques.
- Finally, all three hash tables are displayed.

3. Important Formulas

1. Hash Function

\[h(key)=key \bmod size \]

Calculates the original index of an element.

Linear Probing

$$ h_i(key)=(h(key)+i)\bmod size $$

Checks the next position sequentially when a collision occurs.

3. Quadratic Probing

$$ h_i(key)=(h(key)+i^2)\bmod size $$

Checks positions using square increments to reduce primary clustering.

4. Comparison of Hashing Techniques

Feature	**Separate Chaining**	**Linear Probing**	**Quadratic Probing**
Collision handling	Uses lists	Sequential checking	Square increments
Storage	Multiple elements per index	One element per index	One element per index
Extra memory	Required	Not required	Not required
Clustering	Less primary clustering	Primary clustering	Reduces primary clustering
Implementation	Easy	Easy	Moderate

5. Error Table

Error	**Reason**	**Solution**
IndexError	Accessing an invalid index	Use the modulo operator
TypeError	Passing an incorrect data type	Pass integer keys

NameError	Using an undefined variable	Declare the variable
IndentationError	Incorrect indentation	Maintain proper spacing
SyntaxError	Missing colon or brackets	Correct the syntax
Hash Table Full	No empty position available	Increase the table size
Collision	Two keys produce the same index	Apply collision-handling techniques
Quadratic Probing Failure	No empty position found in the probe sequence	Increase table size or use a suitable table size